

Module 01: Process Concepts, Process States & PCB

1. Process Concept: Program vs Process

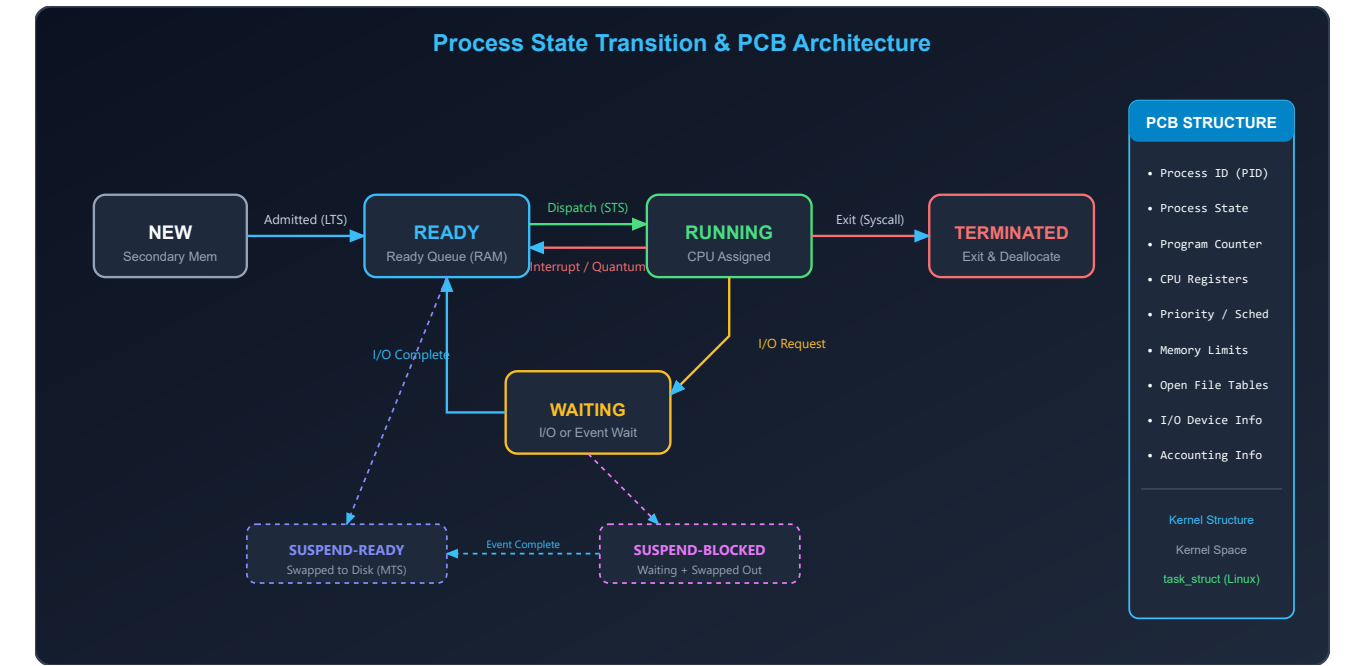
Program (Passive): Disk par stored static machine binary code file hai.

Process (Active): Program in execution. Iske paas memory address space, allocated CPU registers, stack, aur heap hota hai.

Process Address Space Layout

Memory Segment	Description	Growth Direction
Stack	Local variables, function return addresses, parameters	High to Low Memory (Downwards)
Heap	Dynamic runtime allocation (malloc, calloc, new)	Low to High Memory (Upwards)
Data	Global and static variables (Initialized .data and Uninitialized .bss)	Fixed at launch
Text	Compiled read-only executable instructions	Fixed

2. Process State Transition Model



Key Transition Rules:

1. New → Ready:

Long-Term Scheduler (LTS) dwara admission.

2. Ready → Running:

Short-Term Scheduler (STS) / Dispatcher allocation.

3. Running → Ready:

Timer interrupt (Preemption) ya higher priority process arrival.

- 4. **Running** → **Waiting**: I/O request ya event wait system call.
- 5. **Waiting** → **Ready**: I/O completion interrupt (Directly running nahi ja sakta!).

3. Process Control Block (PCB) Fields

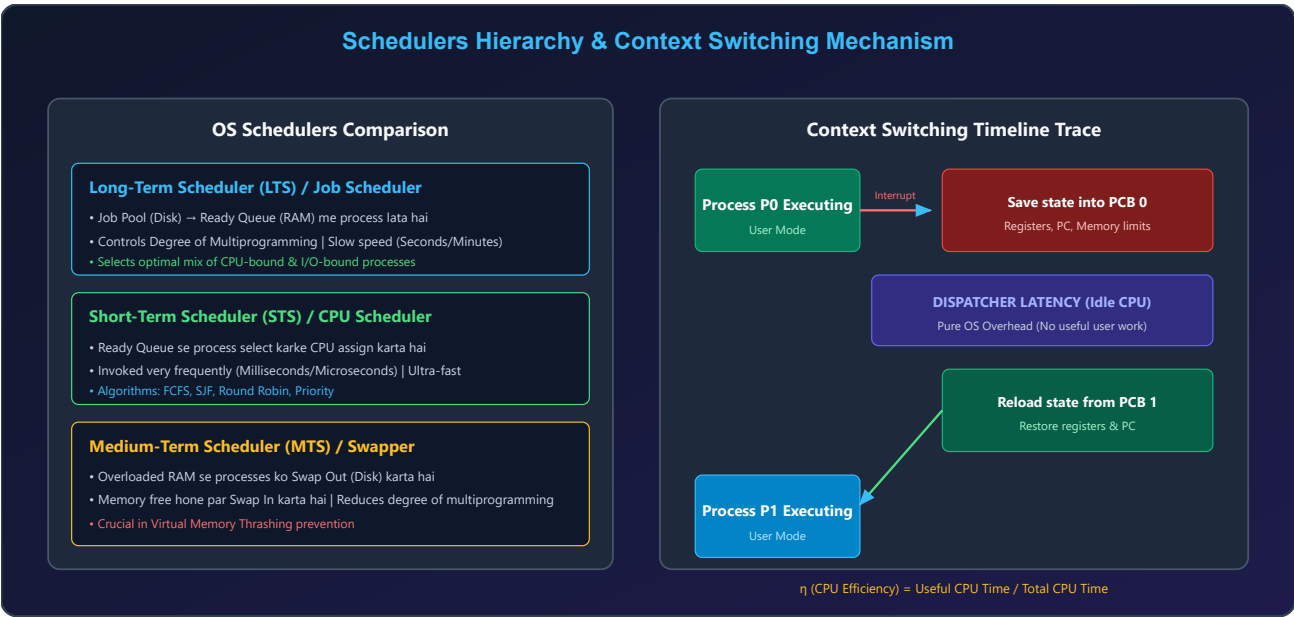
PCB Field	Purpose in OS Kernel
PID / PPID	Unique process identifier and Parent process ID
Process State	Ready, Running, Waiting, Terminated
Program Counter	Next instruction address to be executed on CPU resume
CPU Registers	Saved hardware registers context during preemption
Memory Limits	Base/Limit registers or Page Directory base register (CR3)
File Descriptors	Array of open file handles and I/O sockets

Module 02: Schedulers, Dispatcher & Context Switching

1. OS Schedulers Comparison

Feature	Long-Term Scheduler (LTS)	Short-Term Scheduler (STS)	Medium-Term Scheduler (MTS)
Other Name	Job Scheduler	CPU Scheduler	Swapper
Selection Path	Disk Pool → Ready Queue	Ready Queue → CPU	RAM ↔ Disk (Swap space)
Frequency	Very low (Minutes)	Extremely high (Milliseconds)	Moderate (Seconds)
Speed	Slowest	Fastest	Medium
Multiprogramming	Controls degree directly	No control over degree	Reduces degree of multiprogramming

2. Schedulers & Context Switch Flow



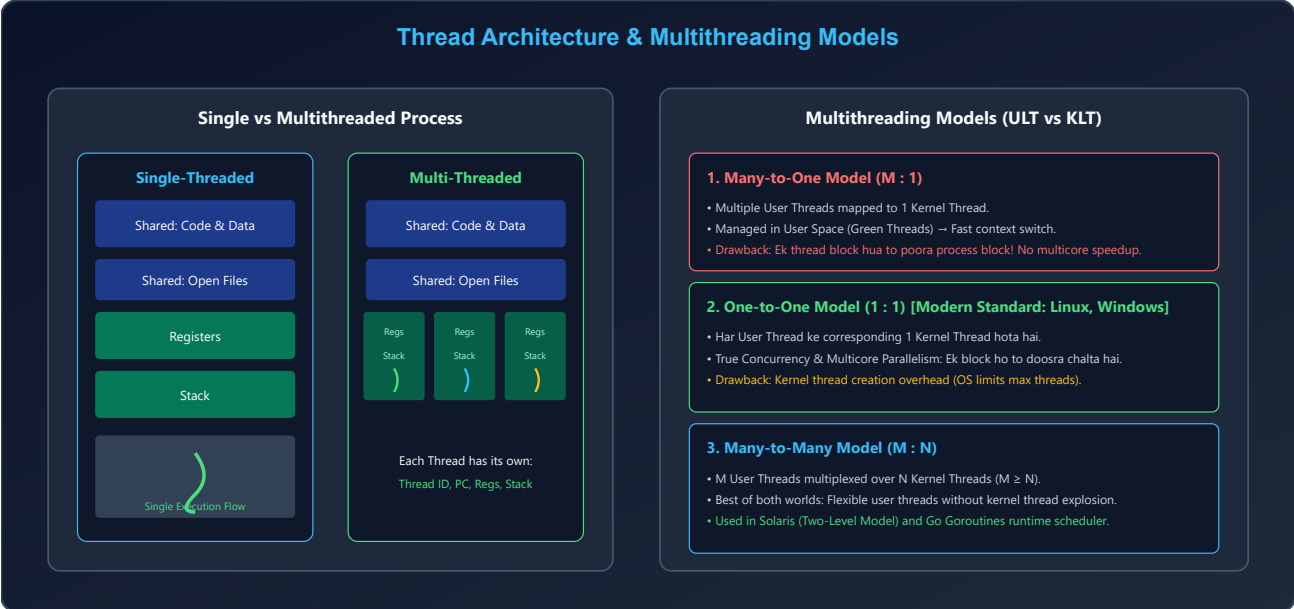
3. Context Switching & Efficiency

Dispatcher Overhead: CPU context switch ke dauran idle rehta hai.

CPU Efficiency (η) = Useful Time / (Useful Time + Context Switch Overhead)

Module 03: Threads & Multithreading Models

1. Single vs Multithreaded Process Architecture



2. ULT vs KLT Comparative Table

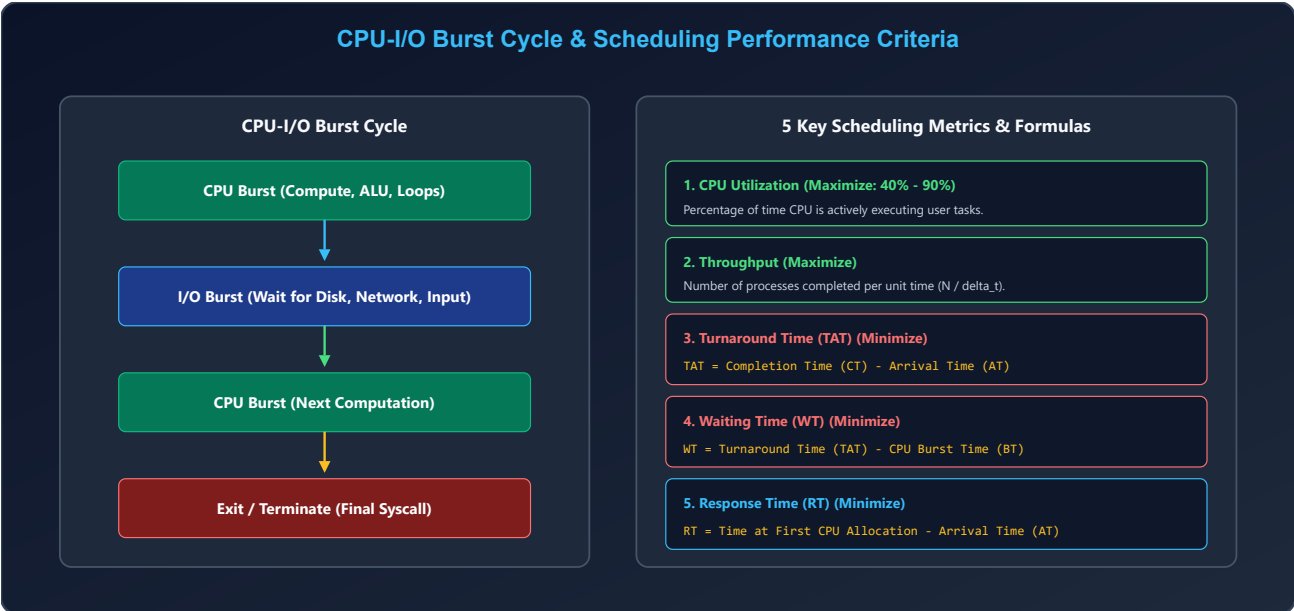
Feature	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Managed by	User-level thread library	Operating System Kernel
Context Switch	Fast (No kernel trap)	Slower (Privilege switch Ring 3 → 0)
System Call Blocking	Entire process is blocked	Only calling thread is blocked
Multicore Parallelism	No (Cannot utilize multiple CPUs)	Yes (Independent core scheduling)

3. Multithreading Models

- Many-to-One (M:1):** Fast, but blocking one blocks all.
- One-to-One (1:1):** High concurrency, modern OS standard (Linux, Windows).
- Many-to-Many (M:N):** Hybrid model, optimal thread multiplexing (Go Goroutines).

Module 04: CPU Scheduling Concepts & Performance Criteria

1. CPU-I/O Burst Cycle & Scheduling Metrics



2. Golden Mathematical Formulas

Metric Name	Mathematical Formula	Optimization Objective
Turnaround Time (TAT)	$TAT = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$	MINIMIZE
Waiting Time (WT)	$WT = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$	MINIMIZE
Response Time (RT)	$RT = \text{First Response Time} - \text{Arrival Time (AT)}$	MINIMIZE
Throughput	$\text{Completed Processes} / \text{Total Time}$	MAXIMIZE
CPU Utilization	$(\text{Active CPU Time} / \text{Total Time}) * 100\%$	MAXIMIZE

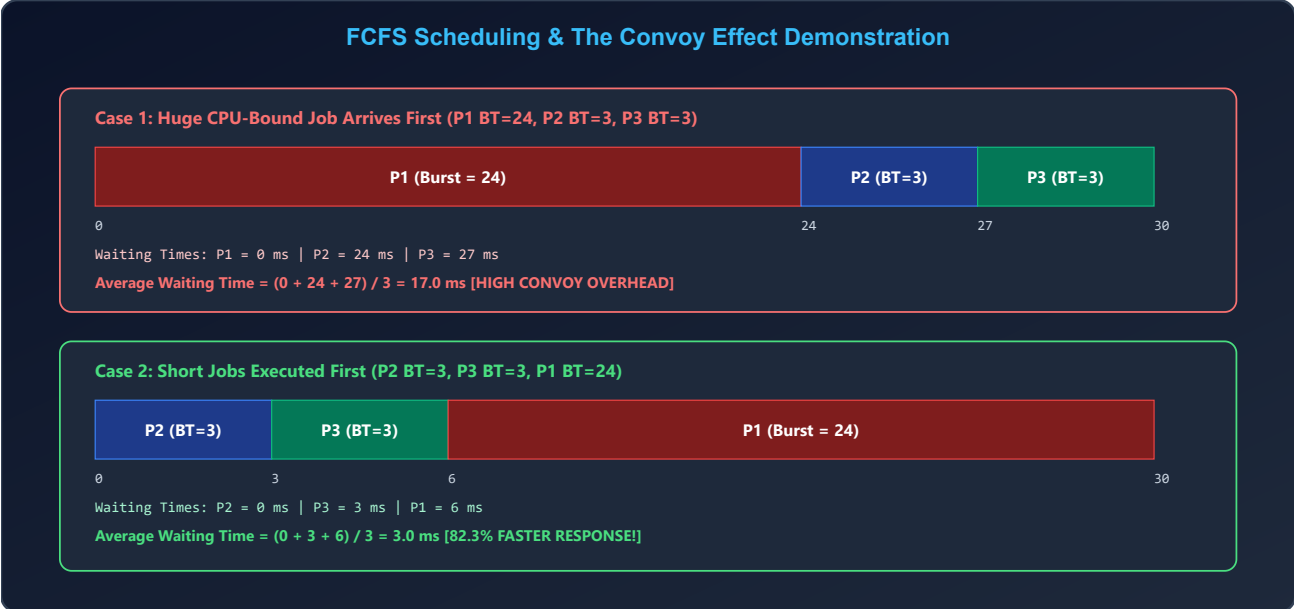
3. Preemptive vs Non-Preemptive Summary

Non-Preemptive: Process willingly CPU release karta hai (I/O or Terminate). $RT == WT$.

Preemptive: OS timer interrupt ya higher priority aane par forcibly CPU preempt kar sakta hai.

Module 05: FCFS Scheduling & The Convoy Effect

1. Convoy Effect Comparison



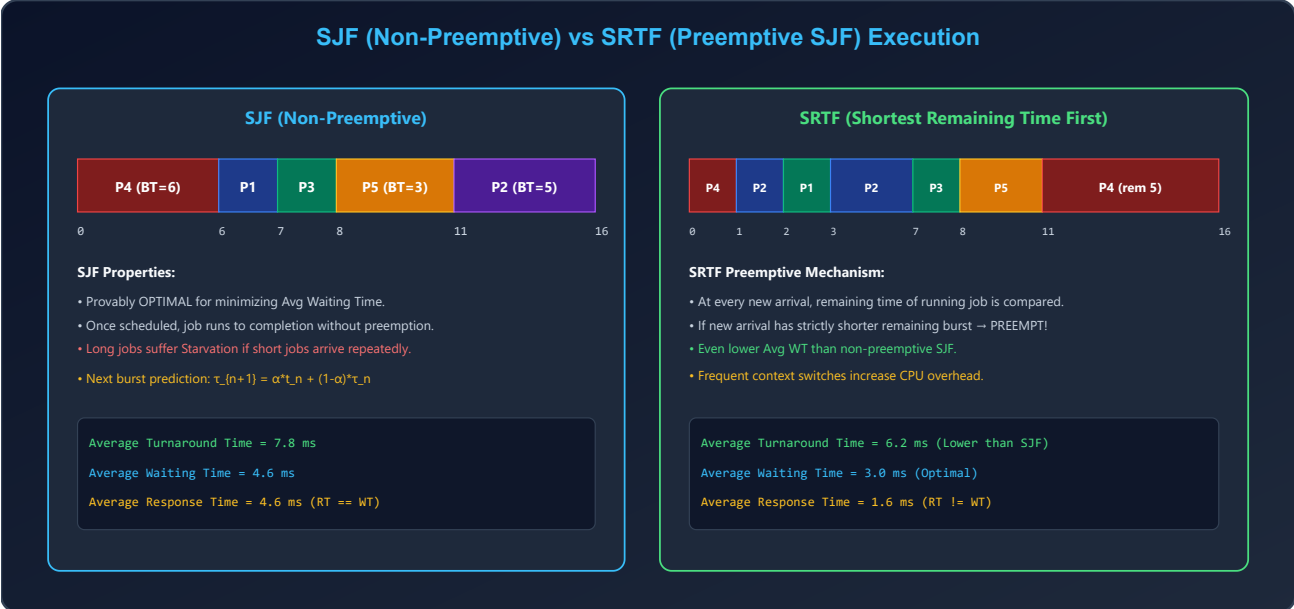
2. Solved Numerical Table (Arrival Time != 0)

Process	AT	BT	CT	TAT (CT - AT)	WT (TAT - BT)
P1	0	4	4	4	0
P2	1	3	7	6	3
P3	2	1	8	6	5
P4	3	2	10	7	5
P5	4	5	15	11	6

Results:
Average TAT = $34 / 5 = 6.8$ ms
Average WT = $19 / 5 = 3.8$ ms

Module 06: SJF (Non-Preemptive) & SRTF (Preemptive SJF) Execution

1. Execution Comparison & Timelines



2. Exponential Smoothing Burst Prediction

$$\tau_{n+1} = \alpha \tau_n + (1 - \alpha) \tau_n$$

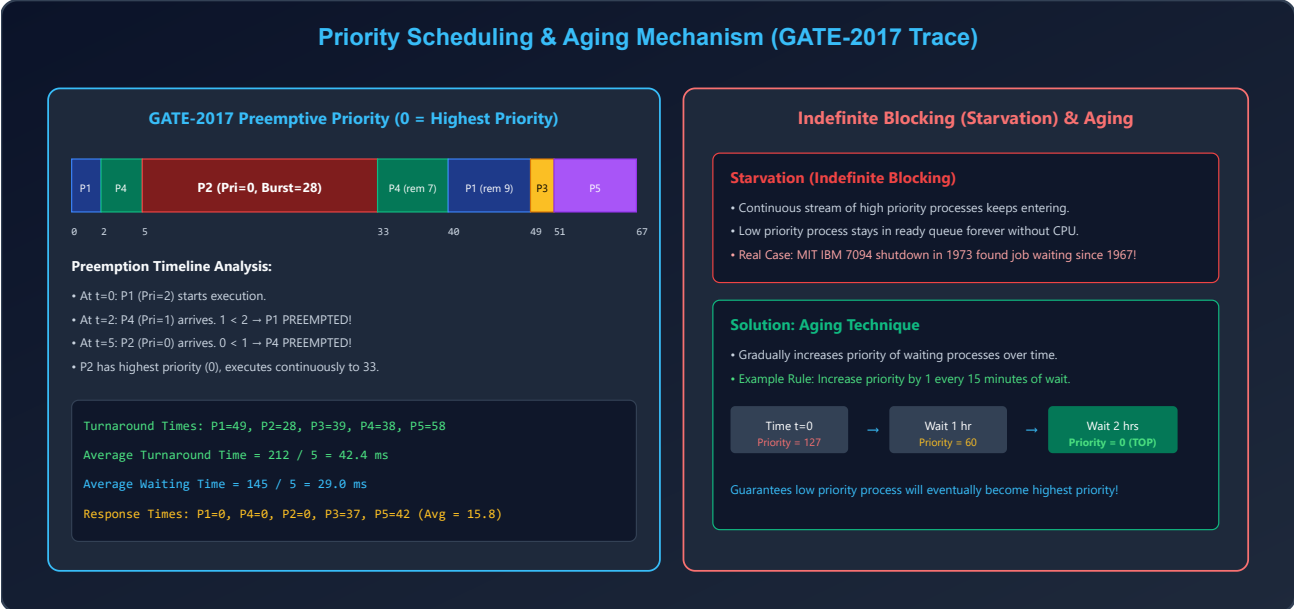
- τ_n : Actual length of n-th CPU burst
- τ_n : Predicted length of n-th CPU burst
- α : Smoothing weight factor ($0 \leq \alpha \leq 1$)

3. Summary Comparison

Metric	Non-Preemptive SJF	Preemptive SJF (SRTF)
Preemption	No	Yes (At each arrival)
Avg WT	4.6 ms	3.0 ms (Lower / Superior)
Response Time	RT == WT	Significantly faster response
Context Switch Overhead	Low	High (Frequent preemption)

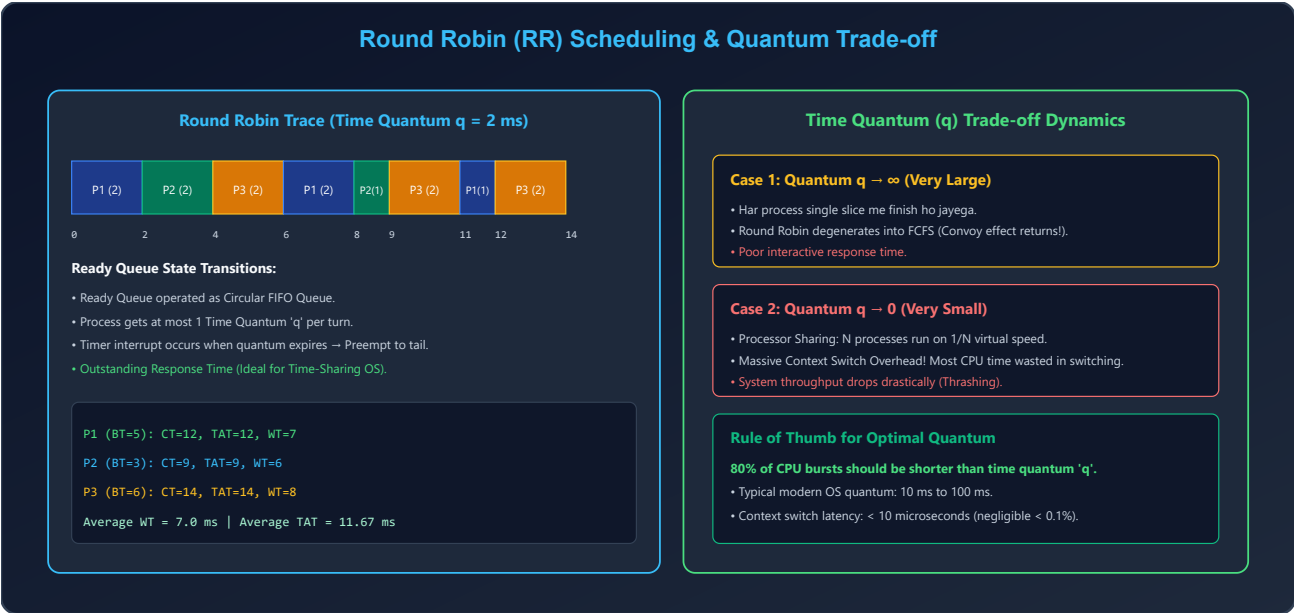
Module 07: Priority Scheduling & Aging Mechanism

1. GATE-2017 Trace & Aging Architecture



Module 08: Round Robin (RR) Scheduling & Quantum Trade-off

1. Execution Flow & Quantum Trade-off



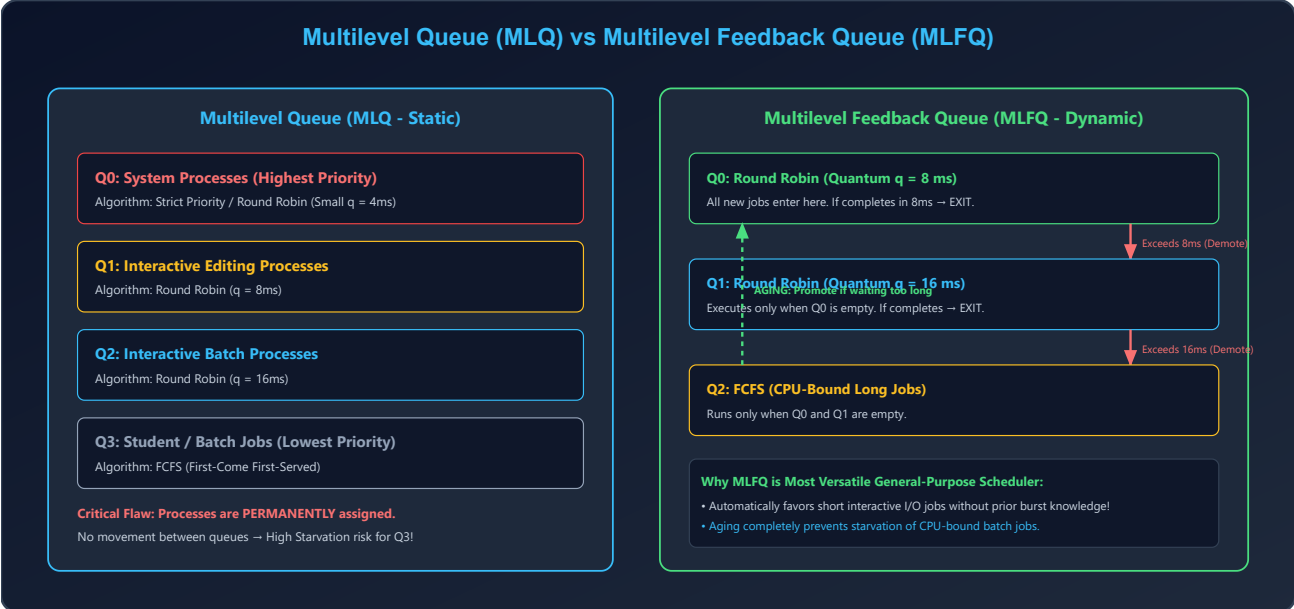
2. Solved Numerical Results ($q = 2\text{ ms}$)

Process	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)	RT
P1	0	5	12	12	7	0
P2	0	3	9	9	6	2
P3	0	6	14	14	8	4

Average TAT = 11.67 ms | Average WT = 7.0 ms | Average RT = 2.0 ms

Module 09: Multilevel Queue & Multilevel Feedback Queue

1. Static MLQ vs Dynamic MLFQ Comparison

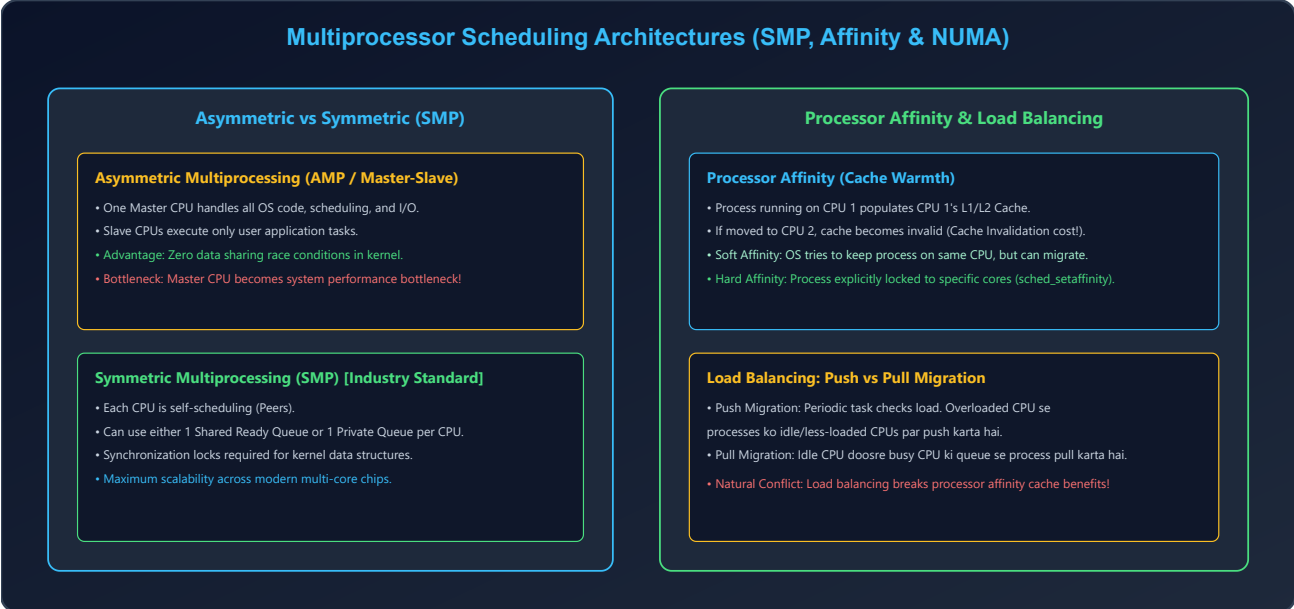


2. Key Differences

Feature	Multilevel Queue (MLQ)	Multilevel Feedback Queue (MLFQ)
Process Movement	None (Permanently bound to 1 queue)	Dynamic promotion and demotion
Prior Knowledge Needed	Process category must be known beforehand	No prior knowledge needed (Self-adjusting)
Starvation Risk	High for lower queues	None (Aging moves old jobs up)
Complexity	Low	High (Multiple configurable parameters)

Module 10: Multiprocessor Scheduling Architectures

1. SMP, Processor Affinity & NUMA Architecture



2. Key Concepts Summary

Concept	Definition & Role
Asymmetric (AMP)	Master CPU handles OS/IO; Slaves run user code only.
Symmetric (SMP)	All CPUs are peers; self-scheduling.
Processor Affinity	Process prefers to run on same CPU to exploit warm L1/L2 cache.
Push Migration	Periodic task redistributes load from overloaded to idle cores.
Pull Migration	Idle core pulls waiting tasks from busy core run-queues.

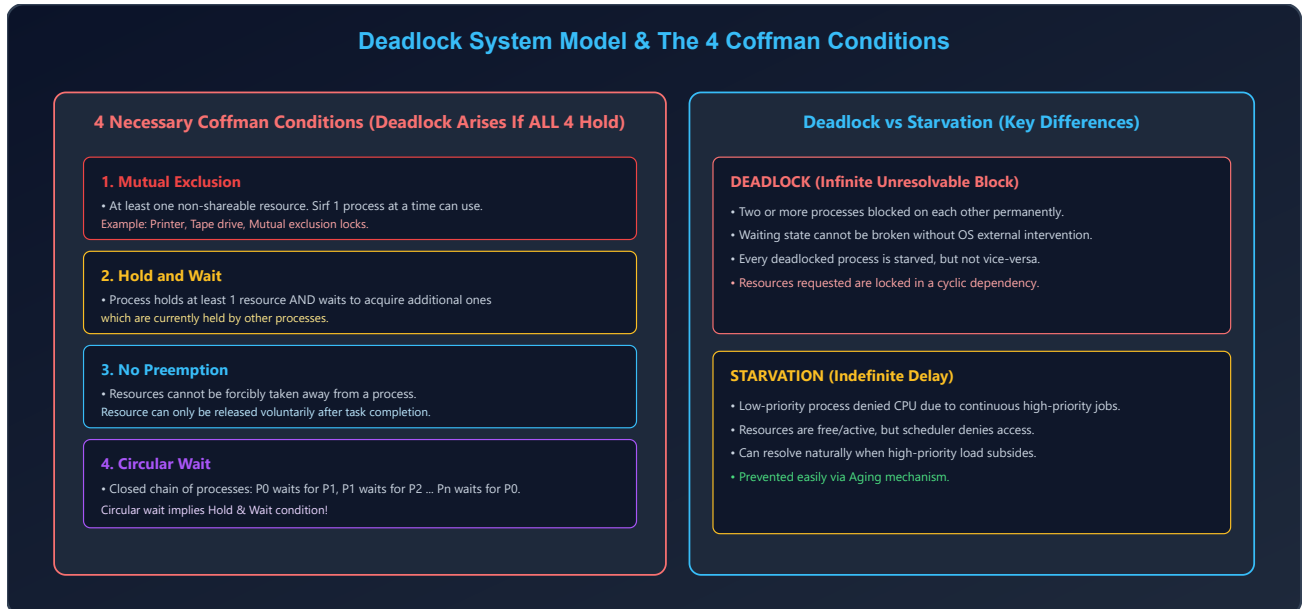
Module 11: Deadlock System Model & 4 Coffman Conditions

AKTU BCS401

Unit 3

Deadlocks

1. The 4 Coffman Conditions & Starvation Comparison

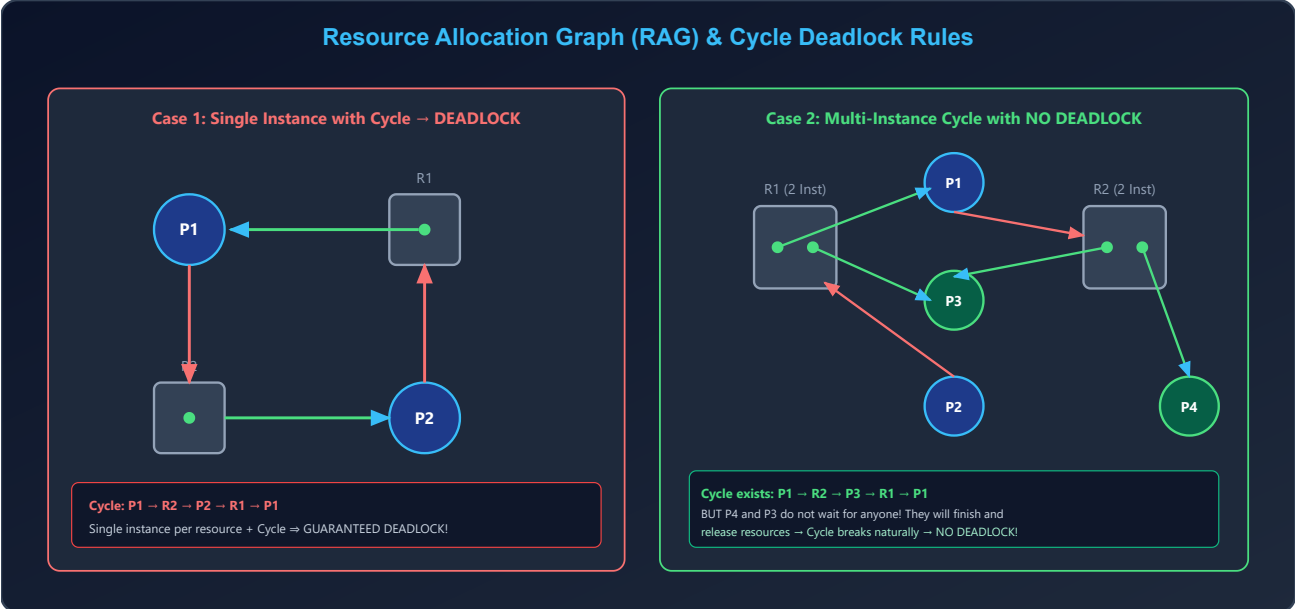


2. The 4 Necessary Conditions

1. **Mutual Exclusion:** Exclusive, non-shareable resources.
2. **Hold and Wait:** Holding 1+ resources while waiting for others.
3. **No Preemption:** Resources cannot be forcibly revoked.
4. **Circular Wait:** Closed loop of waiting processes.

Module 12: Resource Allocation Graph (RAG) & Cycle Rules

1. Cycle Analysis in Single vs Multiple Instance RAG



2. Golden Rules Table

Resource Instance Type	Cycle Present?	Deadlock Status
Any	NO	No Deadlock (100% Safe)
Single-Instance (1 per type)	YES	Deadlock GUARANTEED
Multiple-Instance (> 1 per type)	YES	May or May Not be in Deadlock

Module 13: Deadlock Prevention & Havender's Algorithm

1. Prevention Strategies & Resource Ordering Trace

Invalidating the 4 Coffman Conditions

1. Mutual Exclusion Denial

- Make resources shareable (Read-only files). Spooling for printers.
- Impractical for inherently non-shareable resources (Mutexes, Tape).

2. Hold & Wait Denial (Conservative Protocol)

- Protocol A: Request ALL required resources before execution starts.
- Protocol B: Release all held resources before requesting new ones.
- Severe Drawback: Extremely low resource utilization & starvation!

3. No Preemption Denial

- If P holding R1 asks for unavailable R2 → R1 is forcibly preempted!
- Only feasible for saveable states (CPU registers, RAM pages), not disks.

4. Circular Wait Denial (Havender's Algorithm)

- Imposes global strict linear ordering on all resources.
- $F: R \rightarrow N$. Process can ONLY request in strictly increasing order!

Havender's Protocol (Gateway Slide 230 Trace)

Mathematical Proof of Circular Wait Denial:

If a cycle existed: $F(R0) < F(R1) < F(R2) < \dots < F(Rn) < F(R0)$.
This implies $F(R0) < F(R0)$, which is a mathematical IMPOSSIBILITY!
⇒ Circular Wait can NEVER form in any state under Havender's rule.
⇒ 100% Deadlock Free by Design!

2. The 4 Elimination Protocols

Coffman Condition	Elimination Protocol	Trade-off / Penalty
Mutual Exclusion	Spooling / Shared read-only	Not possible for dedicated hardware
Hold and Wait	Request all resources at start	Very poor resource utilization & starvation
No Preemption	Forcibly revoke held resources	State corruption risk; limited to CPU/RAM
Circular Wait	Havender's strict linear ordering	Best practical method ; programmer discipline needed

Module 14: Deadlock Avoidance & Banker's Algorithm

1. Safe State & Banker's Trace

State Spaces Relationship

All Operating System States

UNSAFE STATE (Deadlock Possible)

DEADLOCK

SAFE STATE (Guaranteed Safe Sequence Exists)

Deadlock Avoidance keeps system strictly in SAFE STATE!

Banker's Algorithm Step-by-Step Resolution Trace

Available Vector: [3, 3, 2] | Need = Max - Allocation
Test Condition for Process P_i: Need_j ≤ Work (Available)

Step 1: Check P2: Need[1,2,2] ≤ Avail[3,3,2] ⇒ SATISFIED! New Avail = [5, 3, 2]

Step 2: Check P4: Need[0,1,1] ≤ Avail[5,3,2] ⇒ SATISFIED! New Avail = [7, 4, 3]

Step 3: Check P5: Need[4,3,1] ≤ Avail[7,4,3] ⇒ SATISFIED! New Avail = [7, 4, 5]

Step 4: Check P1: Need[7,4,3] ≤ Avail[7,4,5] ⇒ SATISFIED! New Avail = [7, 5, 5]

Step 5: Check P3: Need[6,0,0] ≤ Avail[7,5,5] ⇒ SATISFIED! New Avail = [10, 5, 7]

SAFE SEQUENCE: < P2, P4, P5, P1, P3 > (System is SAFE!)

2. Golden Formula & Safe Sequence

Need Matrix Formula:
$$\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$$

Safe Sequence Found: < P2, P4, P5, P1, P3 >

Module 15: Deadlock Detection & Recovery Strategies

1. Wait-For Graph (WFG) & Recovery Flow

Deadlock Detection (Wait-For Graph) & Recovery Protocols

Wait-For Graph (WFG - Single Instance)

```
graph LR; P1((P1)) --> P2((P2)); P2 --> P3((P3)); P3 --> P4((P4)); P4 --> P1
```

Resource Nodes Collapsed!
Pi → Pj edge indicates Pi is waiting for resource held by Pj.
Cycle detection in WFG runs in $O(V^2)$ via DFS/Tarjan.

Deadlock Recovery Strategies

1. Process Termination

- Abort All Deadlocked Processes: Expensive, but guarantees recovery.
- Abort One-by-One: Abort 1 process, rerun detection. Repeat until free.
- Victim Selection: Lowest CPU time spent, lowest resources held, lowest priority, batch vs interactive.

2. Resource Preemption (Gateway Slide 260 Trace)

- Selecting a Victim: Minimize cost of preempted resources.
- Rollback: Roll back victim process to previous safe checkpoint. (Total rollback = abort and restart; Partial = checkpoint restart).
- Starvation Issue: Same process must NOT be picked as victim repeatedly!
- Solution: Include number of rollbacks in victim selection cost function.

2. Recovery Summary Table

Method	Mechanism	Side Effect & Mitigation
Abort All	Kill all deadlocked processes simultaneously	High computation loss
Abort One-by-One	Kill lowest priority victim repeatedly	Detection rerun overhead
Resource Preemption	Revoke resources and rollback to checkpoint	Starvation risk (mitigated by rollback counter)

Module 16: Unit 3 AKTU PYQs & Interview Cheatsheet

AKTU BCS401

Unit 3

Exam Ready

1. Unit 3 Comprehensive Mindmap

